* for int main () for presence of any int there has to be Return 0; at the end.
* empty parentheses actually makes us understand that there is no value that is returned and it can be explicitly mentioned by main (void).
**before main the #define directive is announced
*any mentions with # in the beginning is a PREPROCESSOR COMPILER DIRECTIVE
*stdio.h is actually standard input output, (they are header files containing library functions)
*they are called on with #include preprocessor directive
*the ending brace of the main function is the Logical end of a function.
**No semicolon after a statement outside braces.

BASIC STRUCTURE

Documentation section (set of **comments** giving name, author, details)
Link section((instructions for the compiler to add link functions from the library)
Definition section (**defines** all symbolic constants)
Global declaration (variables that need to used in multiple funtions are mentioned here)

main () function section(contains two parts declaration and executable part

{
Declaration part
Executable part
}

Tokens

A token is the smallest **individual unit** of coding. In the case of the C language, these are called C tokens. (Ref: Page 25 – ANSCII Book)

C Tokens:

1. Keywords – words that are fixed for C and cannot be used by the user (e.g. `float`, `int`, `while` etc.)
2. Identifiers – any words that are used in the program (except keywords)
3. Constants – numbers used in the program (can be decimals)
4. Strings – characters used in the statement (e.g. "cat", "dog")
5. Special Symbols – symbols used for specific purposes in coding (e.g. (), {}, ; etc.)

Operators – symbols of addition, subtraction etc.

Data Types

int	– integers	– 16 bit/32 bit size
char	– characters	– 8 bit
long int	– long integers	– 32 bit size always
long long int		– 64 bit size always
float	– decimal numbers	– 32 bit size
double		– 64 bit
long double		– 80 bit

Computers store the values in binary 0s and 1s. For 9 bits, the first bit is used for the sign and the other 7 bits are used to represent binary numbers from 0 to $2^7 - 1$.

Rules of Identifiers

1)It can only contain alphabets, digits and underscores

2)It cannot **start with a digit**

3)It cannot have **spaces**

4) Cannot use keywords

For constants,

Integer constants: the largest integer saved by a *16 bit machine is 32767

*32 bit machine is 2147483647

*float numbers are also called real constants.(2.165e^2 means 216.5) exponents cant be decimal.

Formatted input

%c----------charactertype (special char)Backslash characters(control character)

%d----------decimal integer type \a---------audible alert

%f----------float form \\---------- display one backslash

%s ----------string type \'----------display one quotation

%e----------exponential form \\"---------- for double quotation

%lf----------Long double type \O----------null

%ld----------Long double or integer type \n----------new line

%i----------intger type \?---------- display ? mark

READING OF A CHARACTER:

char name;

putchar(var); [simply prints a character]

name=getchar(var);

*Here, a character is read, **only a single character** is taken as input.

*It is used on the right side to the variable name.

Identifier of Character & digit: [Comparision of character included]

isalpha()

isdigit()

*these give a Non zero value only if the **character input** is **given for alpha**, a character and **for digit**, **a number**.

*Again these are **Zero** only if **character input** is not **a digit** or **number** respectively.

islower() Lower case letter identifier .

isupper() Upper or Not.

FORMAT USAGE

%-x.y [here **- sign** indicates that it will be in **left alignment** and space will stay on right]

%+x.y[here **+sign** indicates that it will have **right alignment** and space on left]

For %d

("%5de",3) ----------> 3 e [Here **5 acts as the maximum space** that can be taken by the digit. **If more is tried to be printed only the first 5 digits will be taken**.]

("%**4**.**3**d",56)----------> 056 [here **4** acts as a space specifier which gives us the idea of how much space there should be infront of the number (extra space). **3** specifies the maximum number of digits that should be printed. [4 spaces are here where 3 are occupied by the number 2 for 56 and 1 for the 0 and the last space is just a space]]

It is a field width specifier

For %f

("%4.3",5.6767677676)----------> 5.676 [The number after decimal actually restricts the total number of decimal place there should be in a number]

*here decimal point (.) also is a space taker.

*IF WE WRITE 08 it will be counted as invalid as the number is acutally taken as an octal value here. and 8 and 9 are invalid octals.

For %*s

printf("%*s\n", i, "ARKAM");----------> This will have **"i" space** here before ARKAM string. **"*"** is the unknown value the placeholder of which is i. In this way variable space can be anounced for looping purpose and more.

For %*d

The number will be skipped but a input will be taken which will have no use whatsover.

int random = rand() % 100;

For printing of random number whose last 2 digits are taken by modulus of the number.

Operators

[1]Relational

== ----------> will check if operands are equal [result=true if equal}

!= ----------> will check if operand are not equal [result= true]

> ----------> will check if left hand side is bigger than left hand side

<----------> will check if rhs bigger than lhs

>= ----------> will check if result bigger or equal

[2]Logical

&&----------> AND

!!----------> OR

!---------->NOT

[3]Arithmatic operator

There are two types of arithmatic operators. They are

i)Unary operators: Used before the a variable.

UPLUS (+) and UMINUS (-) (both indicates that the numbers are positive or negative.)

ii)Binary operators: (*, +, -) *% gives us the reminder value after division of a number by another number.

USAGE:

* The mixing of variable types during operation is also allowed.

  int op int → int (5 + 10 → 15)

int op float → float (5 + 10.0 → 15.0)

float op float → float (5.0 + 10.0 → 15.0)

* Remainder operator takes the symbol of left operand.

−9%7 = −2

9% − 7 = 2

−9% − 7 = −2

[4]Assignment operators:

*= stores the value in an overall expression.
*all assignment operators are actually **Right associative**.
*They have the **least precedence**.
*Most operators do not modify the value of its operand(s) and produces a new result. However, some operators including simple assignment (=) do modify the value of left operand. Such behavior of operators is called Side Effect.

usage:

1)
```
int i;
float j;
i=72.99 /i=72 as its a variable
j=5       /j=5.00 as its a float value
```

2) The following example shows how the float value is first converted to int to assign to an int and then that int value is converted back to float value, losing the fractional part of the value in process.

```
int i;
float f;
f = i = 72.99;
/ i = 72, f = 72.0 not 72.99/
```

3)
```
int i=1;
i=i+2;  [This can be simplified to i += 2; // i = 3]
```

This is known as **compound assignment**.

List of common arithmetic compounnd assignment operators:



[5]INCREMENT AND DECREMENT OPERATORS

*There are two versions which are Postfix and Prefix.

*i++; → i = i + 1; // (i += 1)

i--; → i = i - 1; // (i -= 1)

*As it is left associative, the Operand is sequenced from the left side.

*Postfix: Here, at first the variable is seen for an opearation and later after that the value is added.

```
int i=4;
        printf("%d",i++);
        \*we will get printed value 4.*\
```

*Prefix: Here, at first the value of variable is added to 1 and then the variable is printed.

```
int i=4
        printf("%d",++i);
        \*we will get printed value 5*\
```

*int a = 0, b = 1, c = 1;

if(a++ && b--)

in this case the value of a is first taken to be 0 and then added later. So the final value of this process inside if will be 0 as a is evaluated to be 0 at first.**FALSE**

int a = 0, b = 1, c = 1;

if(++a && b--)

In the case of this the ++ part will be pre evaluated as it has been used as prefix and so

the final answer will be **TRUE**

[5]Logical operators:

*These types of operators can combine conditions in a condition statement.

*&& is AND operator (this has higher precedence) [BINARY}

*|| is OR operator (this has lower precedence) [BINARY]

* ! is NOT or logical negation. [UNARY}

*Assignment operation can be done in a conditional statement.

*Precedence is **lower** than Arithmatic operators and relattional operators.

***left** associative.

*Here any non zero operators are considered as TRUE

USAGE:

- TRUE=1 and FALSE=0
- NOT(!) !A-> true if A is false (0)
  !A-->false if A is true (1)

- Always work will be done following precedence so, if a conditional statement is written
  int a=5 , b=10;
  if (1<a<10)
  {printf("hello");
  at first as the associativity is from the left, the red part will be executed first and if 1<a is true 1 will be gotten which will then be compared to 10 i.e 1<10. Hello will be printed so brackets should be used and conditions are to be seperated using relational operators.
  Instead we can write if(1<a && a<10)
- In case if a integer or float is dealt with && or ||,

*For any integer value the value will be taken TRUE (ex: 50&&true(1)==1)

* SHORT CIRCUIT: In some actions, sometimes some cases are absolute for which only one part of the code is executed and he rest is not taken into consideration. Activities are started usually from the left side and so the right is ignored.

  *TRUE(1) is ABSOLUTE in an OR operation.

  *FALSE(0) is ABSOLUTE in an AND operation.

* Relational operators can compare mixed type variables too. (between both float and integers and other sorts)
* Following is a "Correct Statement" in C.

$$i < j < k$$

However, it does not check j to be in between i and k. Rather, First checks whether i < j which returns 1(true) or 0(false) .Then it checks whether the previously returned 1 or 0 is less than k, which again returns 1(true) or 0(false) .

Example:

```
int a=50, b=10;
if(a==50 || a>10 && a>b)
```

Only a==50 will be executed and as found TRUE (1) when OR operation occurs anything added to 1 will be 1. So TRUE IS ABSOLUTE.

```
int a=50, b=10;
if(a==40 && a>10 && a>b)
```

only a==40 will executed first and we will get a false and anything Multiplied to 0 is 0 and so the answer will be 0 in the end. So FALSE IS ABSOLUTE.

```
int a=50, b=10;
if(a==50 && a>10 && a>b)
```

in this case as 1 is not absolute..... eventhough its true on the left side, the operation wil be carried out in the right side too.

int a=50, b=10;

if(**a==40** || a>10 && a>b)

in this case too 0 is not absolute and so the right side will be aken into consideration.

[6]EQUALITY OPERATORS:

*used to check the equality between two operands.

*Lower precedence than Relational operators.

* "!=" and "==" are respectively not equal and equal.

USAGE:

* Mixed types can be identified at the same time expressions can also be used

  5=6---> false (0)

  2=2---->true (0)

  5!=6---->false (0)

  5!=2=3-->false(0)
* Dont mix up = and == signs because they will shit on you later.

[7]Comma Operator:

*has the **LOWEST PRECEDENCE** of all the operators.

*It is left associative.

USAGE:

It is evaluated in two steps.

Evaluate expr1 and discard the value

Evaluate expr2 and it becomes the value of the entire equation

This is why expr1 must have side effect, otherwise it is useless.

Example:

i = 1, j = 2, k = i + j → ((i = 1), (j = 2)), (k = (i + j))

So, the key point is that the values of the first two sub-expressions (i = 1 and j = 2) are computed but discarded, and only the value of the last expression (k = i + j) is kept and returned as the result.

Precedence [ORDER OF EVALUATION OF OPERATORS]

Associativity [OPERAND GROUPING SEQUENCE]

***Operations inside the Brackets are always done first**

*using plenty of brackets (parentheses) maybe be best if you are in doubt about Precedence and associativity

- *Left Associative:When operations Grouped from the Left to right

  +, -, *, / are Left Associative.
- Right Associative: When done from the Right.

  UPLUS and UMINUS are right Asscociative.

  -+t --> -t (actual sense)

Precedence & associativity table

Precedence	Name	Symbol(s)	Associativity
1	increment (postfix)	++	left
	decrement (postfix)	--	
2	increment (prefix)	++	right
	decrement (prefix)	--	
	UPLUS	+	
	UMINUS	-	
3	multiplicative	* / %	left
4	additive	+ -	left
5	assignent	= *= /= %= += -=	right

*There might be problematic situations where the computer might not be able to understand which is to be implemented first. Here the computer will just take the route which suits it more in terms of memory reachability.

```
int i=10,
i=(i*2)+(i/2);
```

Here, it is impopssible to tell which one will be implemented by the computer.

*Sometimes the result might also be **changed**.

In most cases, this evaluation order does not matter, but it might change results in case of operators with side effects.

```
int i = 2;
int j;
j = i * i++; → j = (i) * (i++);
```

In the code above, value of j can either be 4 or 6 depending on the implementation of the compiler, so it is an **undefined behavior.**

SELECTION STATEMENT

There are mainly 2 types of selection statements:

i) "if" statement

ii)"switch" statement

iii)temary/conditional operator(?:)

1IF STATEMENT:

*Can execute more than one statement which should end with ";"

*Compound statements can be used in loops and othe places too.

USAGE:

* if (expression) statement {Here if the EXPRESSION in the statement not 0 then the next statement will be executed.
* NESTED IF statements are If statements which can be used

```
if (i > j){
        {if (i > k)
        max = i;
        else
        max = k;}
else if (j > k)
max = j;
else
max = k;}
```

* **Cascaded "if" statement:**

  Best way to test a series of condition is to use cascaded if statements, meaning another

if statement will be nested inside first if statement's else clause and so on.

```
if (n < 0)
printf("n is negative\n"); else
if (n == 0)
printf("n is zero\n"); else
printf("n is positive\n");
```

*Again, it can be nested like if and then else if and the else and so on

[2]*SWITCH STATEMENT:*

*it can **only check EQUALITY.**

*not more than **one condition** can be added to this statement.

*around the blocks of action no braces have to be used.

*the name of the case itself is known as **case label** usually used as integers.

*CASE LABELS END WITH A **COLON(:)**

*Break statement has to be added as required after every cases if multiple cases are not to be runned simultaneously as all of it will run if "break;" statement is not used.

*DEFAULT case has to be added if the None of the case values actually match.

* **Controlling Expression "**switch(controlling statement)**"**

  *Must be an integer/enumerable expression

  *Characters are treated as integers in C (Can be used)

  *Floating point numbers and strings are not allowed

* **Constant Expression "**case controlling statement: statements"

  *Can be evaluated at compile time.

  *Similar to ordinary expression but cannot contain variables or function calls

Duplicate case labels are not allowed

*The order of cases does not matter

*Use of constants and proprocessor constants are fine.

--->5 or 5 + 10 is allowed

----->n + 10 is not allowed (given n is a variable)

* **Statement**

*Any number of statements are allowed

*No braces required around statement

*The last statement is usually a break.

USAGE:

* Several case labels may precede same group of statements.

```
switch (grade) {
case 4:
case 3:
case 2:
case 1: printf("Passing");
break;
case 0: printf("Failing");
break;
default: printf("Illegal grade");
break;
```

* Several cases labels can be put on the same lines to save space

```
switch (grade) {
case 4: case 3: case 2: case 1:
printf("Passing"); break;
case 0: printf("Failing");
break;
```

```
default: printf("Illegal grade");
break;
```

* If default case is missing and controlling expression is 0, control passes to the next function after switch statement.

[3]Conditional statement:(?:)

*An operation that allows an expression to produce one of the two values depending on the condition.

*Operands can be of any type and it requires 3 operands, hence it is a ternary operator.

USAGE:

i)expr1 ? expr2 : expr3

If expr1 is 1(true) , then expr2 is evaluated, otherwise expr3 is evaluated and the evaluated value becomes the value of the entire expression.

ii) It can exchange if and else statement in some cases. For example:

```
if(x>0){
        flag=0;
}                       can be exchanged by          flag=(x<0) ? 0 : 1
else{
flag=1;
}
```

Iteration statements

An iteration statement also known as a "Loop" is a statement whose job is to execute some other statement (loop body) based on some "controlling expressions".

*As long as controlling expression is true (1) it runs.

*It breaks when controlling exp. is 0.

TERMINATION OF LOOPS:

**The loop can be ended from anywhere between if we use the break statement.

** An if statement can judge is the loop was ended prematurely.

*break can escape only one level of nesting in nested statements.

*Usage of **continue**

```
int n = 0, sum = 0, i;
while (n < 10) {
scanf("%d", &i);
if (i <= 0)
continue;
sum += i;
n++;
/* continue jumps to here */
}
```

3 types of loops:

1)while: Controlling exp. is tested BEFORE execution of loop.

2)do : Controlling exp. is tested AFTER execution of loop.

3)for : *It has multi purpose. Assignment(1); Condition(2) ;Increment/Decrement(3)

*when loop is deterministic.

1while:

*form: while (expression) statement

*loop continues until cont. exp. is not 0. (non-zero--->execution--->check cont exp---> non zero (redo)/ zero (dead))

USAGE:

*
```
int i=1;
while(i<n){    //controlling expression
i=i*2;          // loop body
```

*Can use compound statement.

*Always using braces is not required if there is only one statement.

*Body of while loop will not be chekced if the condition is false to begin with.

*Infinity loop & its solution: Loop should be terminated so that it does not turn into an infinity loop and to terminate the loop inside the loop itself we have to use (break, goto, return) or call a function to terminate the program.

[2]do statement:

*This statement executes the loop body first, then evaluates controlling expression.

*The loop body is executed at least once.

*work sequence:

i)EXECUTE FIRST BITCHHHH----->eh condition not match--->ig we can end here then lol.

ii)EXECUUTEEEEE------->ok condtion working----->LEZGOOO--->until it doesnt give false.

USAGE:

i)Usage format: **do statement while** (expression);

ii)A simple code using do statement to find the number of digits in a number.

```
#include <stdio.h>
main (void)
{
```

```
int n,a,div_count ;
printf("Enter a non negative number:");
scanf("%d",&n);
 div_count=0;
 do{
       n/=10;
       div_count++;
       a=n;
   }while (a>0);
printf("%d", div_count);
}
```

[3]for statement:

*Ideal for looping where there are counting variables (deterministic loop count)

USAGE:

* for(**exp 1**, **exp 2,** **exp3**)
  initialization **Cont. expn** **operation**
* For some rare cases "while" can replace the for statement
  initialization;
  while(**controlling expression**){
  statement;
  operation;}
* C99 allows declaration of variabless inside the braces where the value is only usable inside the loop. Here more than one variable can be announced with the use of coma operator.
* At the same time more than one initialization or several increments can be made. Multiples in only **FIRST** and **THIRD** expression.

Nested Looping:

* for obtaining the solution of the star problems:




```
#include <stdio.h>
int main(){
int i, n, j;
scanf("%d",&n);
   for(i=0;i<n;i++){
      for(j=i;j<n;j++)
         printf("*");
   printf("\n");
   }
}
```

* 


```
#include <stdio.h>
int main(){
int i, n, j;
scanf("%d",&n);
   for(i=n ;i>0;i++){
      for(j=n-i ;j<=n; j++)
         printf("*");
```

```
    printf("\n");
    }
}
```